



Jerry Lin <jerry@museder.com>

[WordPress Plugin Directory] Review in Progress: Museder RestoreOne

WordPress.org Plugin Directory <plugins@wordpress.org>

2026年4月2日 凌晨3:20

收件者: Jerry Lin <jerry@museder.com>

🙌 Thank you for the changes "artherslin"!

Your plugin is not yet ready to be approved, you are receiving this email because the volunteers have manually checked it and have found some issues in the code / functionality of your plugin.

Please check this email thoroughly, address any issues listed, test your changes, and upload a corrected version of your code if all is well.

List of issues found

Nonces and User Permissions Needed for Security

Please add a nonce check to your input calls (\$_POST, \$_GET, \$REQUEST) to prevent unauthorized access.

If you use wp_ajax_ to trigger submission checks, remember they also need a nonce check.

👮 Checking permissions: Keep in mind, a nonce check alone is not bulletproof security. Do not rely on nonces for authorization purposes. When needed, use it together with current_user_can() in order to prevent users without the right permissions from accessing things they shouldn't.

Also make sure that the nonce logic is correct by making sure it cannot be bypassed. Checking the nonce with current_user_can() is great, but mixing it with other checks can make the condition more complex and, without realising it, bypassable, remember that **anything** can be sent through an input, **don't trust any input**.

Keep performance in mind. Don't check for post submission outside of functions. Doing so means that the check will run on every single load of the plugin, which means that every single person who views any page on a site using your plugin will be checking for a submission. This will make your code slow and unwieldy for users on any high traffic site, leading to instability and eventually crashes.

The following links may assist you in development:

- <https://developer.wordpress.org/plugins/security/nonces/>
- <https://developer.wordpress.org/plugins/javascript/ajax/#nonce>
- <https://developer.wordpress.org/plugins/settings/settings-api/>

From your plugin:

```
includes/class-restore-handler.php:1238 Museder_Restoreone_Restore_Handler::chunk_prepare() [classMethod]
No nonce check found validating input origin on lines 1238-1284
# ↳ Line 1245: $filename = sanitize_file_name( wp_unslash( $_POST['filename'] ) );
# ↳ Line 1251: $filesize = absint( wp_unslash( $_POST['filesize'] ) );
# ↳ Line 1257: $chunk_size = absint( wp_unslash( $_POST['chunk_size'] ) );
# ↳ Line 1263: $total_chunks = absint( wp_unslash( $_POST['total_chunks'] ) );
includes/class-ui.php:828 Museder_Restoreone_UI::handle_delete_backup() [classMethod] No nonce check
found validating input origin on lines 828-838
```

```
# ↳ Line 832: $filename = isset( $_POST['filename'] ) ? sanitize_text_field( wp_unslash( $_POST['filename'] ) ) :
'';
includes/class-ui.php:597 Museder_Restoreone_UI::ajax_continue_backup_job() [classMethod] No nonce check
found validating input origin on lines 597-607
# ↳ Line 601: $job_id = isset( $_POST['job_id'] ) ? sanitize_text_field( wp_unslash( $_POST['job_id'] ) ) : '';
includes/class-ui.php:1071 Museder_Restoreone_UI::handle_backup_download() [classMethod] No nonce check
found validating input origin on lines 1071-1095
# ↳ Line 1078: $file = isset( $_GET['file'] ) ? sanitize_file_name( wp_unslash( $_GET['file'] ) ) : '';
# ↳ Line 1089: $expires = isset( $_GET['expires'] ) ? absint( wp_unslash( $_GET['expires'] ) ) : 0;
# ↳ Line 1090: $token = isset( $_GET['token'] ) ? sanitize_text_field( wp_unslash( $_GET['token'] ) ) : '';
includes/class-restore-handler.php:1543 Museder_Restoreone_Restore_Handler::chunk_abort() [classMethod]
No nonce check found validating input origin on lines 1543-1551
# ↳ Line 1548: $session_id = isset( $_POST['session_id'] ) ? sanitize_text_field( wp_unslash(
$_POST['session_id'] ) ) : '';
```

... out of a total of 6 incidences.

Please, make sure that the nonce logic is correct. It's important to be cautious when structuring conditional checks around nonces.

```
includes/ai/class-ai-rest-controller.php:79 if ( ' === $nonce || ! wp_verify_nonce( $nonce, 'wp_rest' ) ) {
```

> Don't use || It must occur all times

Generic function/class/define/namespace/option names

All plugins must have unique function names, namespaces, defines, class and option names. This prevents your plugin from conflicting with other plugins or themes. We need you to update your plugin to use more unique and distinct names.

A good way to do this is with a prefix. For example, if your plugin is called "Museder RestoreOne" then you could use names like these:

- function musere_save_post(){ ... }
- class MUSERE_Admin { ... }
- update_option('musere_options', \$options);
- add_shortcode('musere_shortcode', \$callback);
- register_setting('musere_settings', 'musere_user_id', ...);
- define('MUSERE_PLUGIN_DIR', plugin_dir_path(__FILE__));
- global \$musere_options;
- add_action('wp_ajax_musere_save_data', ...);
- namespace artherslin\musederrestoreone;

Disclaimer: These are just examples that may have been self-generated from your plugin name, we trust you can find better options. If you have a good alternative, please use it instead, this is just an example.

The prefix should be **at least four (4) characters long** (don't try to use two- or three-letter prefixes anymore). We host almost 100,000 plugins on WordPress.org alone. There are tens of thousands more outside our servers. Believe us, you're likely to encounter conflicts.

You also need to avoid the use of `__` (double underscores), `wp_`, or `_` (single underscore) as a prefix. Those are reserved for WordPress itself. You can use them inside your classes, but not as stand-alone function.

Please remember, if you're using `_n()` or `__()` for translation, that's fine. We're **only** talking about functions you've created for your plugin, not the core functions from WordPress. In fact, those core features are why you need to not use those prefixes in your own plugin! You don't want to break WordPress for your users.

Related to this, using `if (function_exists('NAME')) {` around all your functions and classes sounds like a great idea until you realize the fatal flaw. If something else has a function with the same name and their code loads first, your plugin will break. Using `if-exists` should be reserved for shared libraries only.

Remember: Good prefix names are unique and distinct to your plugin. This will help you and the next person in

debugging, as well as prevent conflicts.

Analysis result:

This plugin is using the prefix "museder" for 152 element(s).

Looks like there are elements not using common prefixes.

```
includes/class-backup-jobs.php:963 add_option($key, $value, '', 'no');
```

```
includes/class-backup-jobs.php:980 add_option($key, $value, '', 'no');
```

Note: Options and Transients must be prefixed.

This is really important because the options are stored in a shared location and under the name you have set. If two plugins use the same name for options, they will find an interesting conflict when trying to read information introduced by the other plugin.

Also, once your plugin has active users, changing the name of an option is going to be really tricky, so let's make it robust from the very beginning.

Example(s) from your plugin:

```
includes/class-backup-jobs.php:963 add_option($key, $value, '', 'no');
```

```
includes/class-backup-jobs.php:980 add_option($key, $value, '', 'no');
```

👉 Continue with the review process.

Read this email thoroughly.

Please, take the time to fully understand the issues we've raised. Review the examples provided, read the relevant documentation, and research as needed. Our goal is for you to gain a clear understanding of the problems so you can address them effectively and avoid similar issues when maintaining your plugin in the future.

Note that there may be false positives – we are humans and make mistakes, we apologize if there is anything we have gotten wrong. If you have doubts you can ask us for clarification, when asking us please be clear, concise, direct and include an example.

📋 Complete your checklist.

✓ I fixed all the issues in my plugin based on the feedback I received and my own review, as I know that the Plugins Team may not share all cases of the same issue. I am familiar with tools such as Plugin Check, PHPCS + WPCS, and similar utilities to help me identify problems in my code.

✓ I tested my updated plugin on a clean WordPress installation with [WP_DEBUG set to true](#).

⚠️ **Do not skip this step.** Testing is essential to make sure your fixes actually work and that you haven't introduced new issues.

✓ I acknowledge that this review will be rejected if I overlook the issues or fail to test my code.

✓ I went to ["Add your plugin"](#) and uploaded the updated version. I can continue updating the code there throughout the review process — the team will always check the latest version.

✓ I replied to this email. I was concise and shared any clarifications or important context that the team needed to know.

I didn't list all the changes, as the team will review the entire plugin again and that is not necessary at all.

 To make this process as quick as possible and to avoid burden on the volunteers devoting their time to review this plugin's code, we ask you to thoroughly check all shared issues and fix them before sending the code back to us. I know we already asked you to do so, and it is because we are really trying to make it very clear.

While we try to make our reviews as exhaustive as possible we, like you, are humans and may have missed things. We appreciate your patience and understanding.

Review ID: R museder-restoreone/artherslin/1Dec25/T11 1Apr26/3.9 (P0TDX263980HGN)

--

WordPress Plugins Team | plugins@wordpress.org

<https://make.wordpress.org/plugins/>

<https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/>

<https://wordpress.org/plugins/plugin-check/>

On Tue, Mar 24, 2026 at 3:13 PM UTC, WordPress.org Plugin Directory <plugins@wordpress.org> wrote:

This is an automated message to confirm that we have received your updated plugin file.

File updated by artherslin, version 2.7.246.

Comment: Dear Plugin Review Team,

Thank you for your continued review of **Museder RestoreOne**. I have uploaded **version 2.7.246**.

This update specifically addresses the WordPress.org review concerns:

- **Guideline 5 / locked functionality:** the free package no longer bundles premium activation, license checks, or separate PRO modules. Optional functionality is only available when a **separate add-on plugin** is installed.
- **Readme disclosure / external services:** the readme now clearly states that the plugin **does not use third-party external services**. It also explains the short, non-blocking **loopback request** to this site's own `wp-cron.php` used to help scheduled tasks run.
- **Readme structure:** the readme now has a **single Changelog** section.
- **REST API permissions:** All routes now use the plugin-prefixed namespace `museder-restoreone/v1`, and the REST endpoints use proper `permission_callback` checks rather than being publicly writable.
- **Security / request handling:** the plugin keeps nonce verification, capability checks, sanitization/validation, direct file access guards, and avoids unsafe direct WordPress bootstrap patterns.

Before upload, I checked the code with **Plugin Check 1.7.0** bundled PHPCS ruleset (`plugin-review.xml`), which reported **0 errors and 0 warnings**.

If anything still needs adjustment, I am happy to follow up.

Best regards,

Jerry Lin

Author of Museder RestoreOne

https://tw.wordpress.org/plugins/files/2025/11/24_15-13-03_museder-restoreone.zip

--

WordPress Plugins Team | plugins@wordpress.org

<https://make.wordpress.org/plugins/>

<https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/>

<https://wordpress.org/plugins/plugin-check/>

On Tue, Mar 17, 2026 at 4:58 PM UTC, WordPress.org Plugin Directory <plugins@wordpress.org> wrote:



Thank you for the changes "artherslin"!

Your plugin is not yet ready to be approved, you are receiving this email because the volunteers have manually checked it and have found some issues in the code / functionality of your plugin.

Please check this email thoroughly, address any issues listed, test your changes, and upload a corrected

version of your code if all is well.

List of issues found

Trialware and Locked Features

Please review your plugin to ensure that it **does not include any locked or restricted built-in functionality**. This is not permitted under the [WordPress.org Plugin Directory Guidelines](#) you agreed to when submitting the plugin.

Guideline 5 — Trialware

Plugins must be fully functional. You may not:

- Lock, disable or limit built-in features behind a license key, trial period, usage limit, time, quota or any other kind of intended restriction.

Even if the locked feature is present in the code "just in case the user upgrades," it's still **not allowed**. Your plugin may point out which features are available through a separated plugin, but that's it. All plugin code hosted on WordPress.org must be **free and fully functional**.

Guideline 6 — Serviceware

Plugins may connect to a **legitimate external service to perform certain functionality**, provided:

- The service performs actual processing on external servers.
- The functionality provided cannot be done locally by the plugin.
- The service is clearly documented in your readme, including **Terms of Use** and **Privacy Policy** links.

For example: a "Spam checker" plugin that connects to a external service to check for spam (and thus uses it to provide that functionality) is generally acceptable. A plugin that simply checks a license key to unlock local features is not.

Ask yourself:

- Does any function **only work after a license check or payment**?
- Is any functionality in the plugin code **disabled or limited** until it's unlocked?
- Are there any limitations on the plugin **after a certain amount of time or usage**?

After excluding functionalities provided by legitimate external services, if the answer is **yes** to any of the above, the plugin **does not comply**.

How to fix it:

- **Remove all license checks** or other mechanisms that control access to features built in in the plugin code.
- **Remove or fully enable** any built in features that are currently locked or limited.
- Make sure external services are compliant and clearly documented.

Important clarification:

WordPress.org is **not a marketplace**. It's a repository for **free, fully functional, GPL-compliant plugins**.

If you are not offering a service and want to offer additional features through a paid version, that code must be:

- **Hosted elsewhere** (e.g., your own website).
- **Not included** in the plugin hosted on WordPress.org.
- **GPL compliant**: Do **not** include any mechanisms that would prevent a plug-in from being used after a license has been checked.

✨ The codebase contains multiple locally-implementable features that are present in this plugin but hard-gated by `Museder_Restoreone_Pro::is_pro_active()` (e.g., free limited to 1 schedule and PRO-only backup options/AI logic).

Undocumented use of a 3rd Party / external service

Plugins are permitted to require the **use of third party/external services** as long as they are clearly **documented**.

When your plugin reach out to external services, you must disclose it. This is true even if you are the one providing that service.

You are required to document it in a clear and plain language, so users are aware of: **what data is sent, why, where and under which conditions**.

To do this, you must update your readme file to **clearly explain that your plugin relies on third party/external services**, and include **at least the following information** for each third party/external service that this plugin uses:

- What the service is and what it is used for.
- What data is sent and when.
- Provide links to the service's terms of service and privacy policy.

Remember, this is for your own legal protection. Use of services must be upfront and well documented. This allows users to ensure that any legal issues with data transmissions are covered.

Example:

```
== External services ==
```

This plugin connects to an API to obtain weather information, it's needed to show the weather information and forecasts in the included widget.

It sends the user's location every time the widget is loaded (If the location isn't available and/or the user hasn't given their consent, it displays a configurable default location).

This service is provided by "PRT Weather INC": [terms of use](#), [privacy policy](#).

Example(s) from your plugin:

```
# Domain(s) not mentioned in the readme file.
```

```
museder-restoreone.php:4 Plugin URI: https://musederlabs.com/restoreone-plugin/
```

```
# Domain(s) mentioned in the readme file. Links to service terms and/or privacy policy not found.
```

```
includes/class-backup-lite-s3-service.php:849 $host = $bucket . '.s3.' . $region . 'amazonaws.com';
```

```
includes/class-backup-lite-s3-service.php:851 $host = 's3.' . $region . '.amazonaws.com';
includes/class-cloud-service.php:220 : sprintf( 'https://s3.%s.amazonaws.com', $settings['s3_region'] );
```

Determine files and directories locations correctly

WordPress provides several functions for easily determining where a given file or directory lives.

We detected that **the way your plugin references some files, directories and/or URLs may not work with all WordPress setups**. This happens because there are hardcoded references or you are using the WordPress internal constants.

Let's improve it, please check out the following documentation:

<https://developer.wordpress.org/plugins/plugin-basics/determining-plugin-and-content-directories/>

It contains all the functions available to determine locations correctly.

Most common cases in plugins can be solved using the following functions:

- For where your plugin is located: `plugin_dir_path()` , `plugin_dir_url()` , `plugins_url()`
- For the uploads directory: `wp_upload_dir()` (*Note: If you need to write files, please do so in a folder in the uploads directory, not in your plugin directories*).

Example(s) from your plugin:

```
includes/class-backup.php:4519 $prefixes[] = wp_normalize_path( trailingslashit( WP_LANG_DIR ) );
includes/class-restore.php:300 $upgrade = rtrim( (string) ABSPATH, '/' ) . '/wp-admin/includes/upgrade.php';
```

Don't Force Set PHP Limits Globally

While many plugins can need optimal settings for PHP, we ask you please not set them as global defaults.

Having defines like `ini_set('memory_limit', '-1');` run globally (like on `init` or in the `__construct()` part of your code) means you'll be running that for everything on the site, which may cause your users to fall out of compliance with any limits or restrictions on their host.

If you must use those, you need to limit them specifically to only the exact functions that require them.

Example(s) from your plugin:

```
includes/class-restore-handler.php:299 set_time_limit(600);
includes/class-backup.php:4037 set_time_limit(0);
```

Review: Missing permission_callback in REST API Route

When using `register_rest_route()` or `wp_register_ability()` to define custom REST API endpoints, it is crucial to include a proper `permission_callback` .

 This callback function ensures that only authorized users can access or modify data through your endpoint.

Code example, checking that the user can change options:

```
register_rest_route( 'museder-restoreone/v1', '/my-endpoint', array(
    'methods' => 'GET',
    'callback' => 'museder-restoreone_callback_function',
```



```
'permission_callback' => function() {
    return current_user_can( 'manage_options' );
}
);
```

Please check the [register_rest_route\(\) documentation](#) and the [current_user_can\(\) documentation](#).

✅ When a permission_callback is NOT Required:

There are valid use cases for public endpoints, such as publicly available data (e.g., posts, public metadata) or endpoints designed for unauthenticated access (e.g., fetching public stats or information).

In these cases, you should use `__return_true` as the `permission_callback` to indicate that the endpoint is intentionally public.

🔒 When a permission_callback IS Required:

For endpoints that involve sensitive data or actions (e.g., getting not public data, creating, updating, or deleting content).

In these cases, you should always implement proper permission checks.

Possible cases found on this plugin's code:

```
includes/pro/ai-controller.php:86 register_rest_route(self::NAMESPACE, '/' . self::BASE . '/nl-command',
['methods' => 'POST', 'callback' => [__CLASS__, 'handle_nl_command'], 'permission_callback' =>
[__CLASS__, 'check_permission']]);
# ↳ Detected: check_permission
# ✨ check_permission checks manage_options but only requires a non-empty nonce value and does not
actually verify it (wp_verify_nonce missing).
includes/pro/ai-controller.php:76 register_rest_route(self::NAMESPACE, '/' . self::BASE . '/smart-schedule',
['methods' => 'GET', 'callback' => [__CLASS__, 'handle_smart_schedule'], 'permission_callback' =>
[__CLASS__, 'check_permission']]);
# ↳ Detected: check_permission
# ✨ check_permission checks manage_options but only requires a non-empty nonce value and does not
actually verify it (wp_verify_nonce missing).
```

Nonces and User Permissions Needed for Security

Please add a nonce check to your input calls (`$_POST`, `$_GET`, `$REQUEST`) to prevent unauthorized access.

If you use `wp_ajax_` to trigger submission checks, remember they also need a nonce check.

👮 Checking permissions: Keep in mind, a nonce check alone is not bulletproof security. Do not rely on nonces for authorization purposes. When needed, use it together with `current_user_can()` in order to prevent users without the right permissions from accessing things they shouldn't.

Also make sure that the nonce logic is correct by making sure it cannot be bypassed. Checking the nonce with `current_user_can()` is great, but mixing it with other checks can make the condition more complex and, without realising it, bypassable, remember that anything can be sent through an input, **don't trust any input**.

Keep performance in mind. Don't check for post submission outside of functions. Doing so means that the check will run on every single load of the plugin, which means that every single person who views any page on a site using your plugin will be checking for a submission. This will make your code slow and unwieldy for users on any high traffic site, leading to instability and eventually crashes.

The following links may assist you in development:

- <https://developer.wordpress.org/plugins/security/nonces/>

- <https://developer.wordpress.org/plugins/javascript/ajax/#nonce>
- <https://developer.wordpress.org/plugins/settings/settings-api/>

From your plugin:

```
includes/class-restore-handler.php:1540 Museder_Restoreone_Restore_Handler::chunk_abort()
[classMethod] No nonce check found validating input origin on lines 1540-1548
# ↳ Line 1545: $session_id = isset( $_POST['session_id'] ) ? sanitize_text_field( wp_unslash(
$_POST['session_id'] ) ) : '';
includes/class-ui.php:1032 Museder_Restoreone_UI::handle_log_download() [classMethod] No nonce check
found validating input origin on lines 1032-1042
# ↳ Line 1039: $log = sanitize_text_field( wp_unslash( $_GET['log'] ?? '' ) );
# ↳ Line 1042: $log = sanitize_text_field( wp_unslash( $_GET['amp;log'] ) );
includes/class-ui.php:1080 Museder_Restoreone_UI::handle_backup_download() [classMethod] No nonce
check found validating input origin on lines 1080-1104
# ↳ Line 1087: $file = isset( $_GET['file'] ) ? sanitize_file_name( wp_unslash( $_GET['file'] ) ) : '';
# ↳ Line 1098: $expires = isset( $_GET['expires'] ) ? absint( wp_unslash( $_GET['expires'] ) ) : 0;
# ↳ Line 1099: $token = isset( $_GET['token'] ) ? sanitize_text_field( wp_unslash( $_GET['token'] ) ) : '';
includes/class-restore-handler.php:1235 Museder_Restoreone_Restore_Handler::chunk_prepare()
[classMethod] No nonce check found validating input origin on lines 1235-1281
# ↳ Line 1242: $filename = sanitize_file_name( wp_unslash( $_POST['filename'] ) );
# ↳ Line 1248: $filesize = absint( wp_unslash( $_POST['filesize'] ) );
# ↳ Line 1254: $chunk_size = absint( wp_unslash( $_POST['chunk_size'] ) );
# ↳ Line 1260: $total_chunks = absint( wp_unslash( $_POST['total_chunks'] ) );
includes/class-ui.php:629 Museder_Restoreone_UI::ajax_cancel_backup_job() [classMethod] No nonce
check found validating input origin on lines 629-639
# ↳ Line 633: $job_id = isset( $_POST['job_id'] ) ? sanitize_text_field( wp_unslash( $_POST['job_id'] ) ) : '';
```

... out of a total of 8 incidences.

Please, make sure that the nonce logic is correct. It's important to be cautious when structuring conditional checks around nonces.

```
includes/ai/class-ai-rest-controller.php:79 if ( '' === $nonce || ! wp_verify_nonce( $nonce, 'wp_rest' ) ) {
```

Generic function/class/define/namespace/option names

All plugins must have unique function names, namespaces, defines, class and option names. This prevents your plugin from conflicting with other plugins or themes. We need you to update your plugin to use more unique and distinct names.

A good way to do this is with a prefix. For example, if your plugin is called "Museder RestoreOne" then you could use names like these:

- function musere_save_post(){ ... }
- class MUSERE_Admin { ... }
- update_option('musere_options', \$options);
- add_shortcode('musere_shortcode', \$callback);
- register_setting('musere_settings', 'musere_user_id', ...);
- define('MUSERE_PLUGIN_DIR', plugin_dir_path(__FILE__));
- global \$musere_options;
- add_action('wp_ajax_musere_save_data', ...);
- namespace artherslin\musederrestoreone;

Disclaimer: These are just examples that may have been self-generated from your plugin name, we trust you can find better options. If you have a good alternative, please use it instead, this is just an example.

The prefix should be **at least four (4) characters long** (don't try to use two- or three-letter prefixes anymore). We host almost 100,000 plugins on WordPress.org alone. There are tens of thousands more outside our servers.

Believe us, you're likely to encounter conflicts.

You also need to avoid the use of `__` (double underscores), `wp_`, or `_` (single underscore) as a prefix. Those are reserved for WordPress itself. You can use them inside your classes, but not as stand-alone function.

Please remember, if you're using `_n()` or `__()` for translation, that's fine. We're **only** talking about functions you've created for your plugin, not the core functions from WordPress. In fact, those core features are why you need to not use those prefixes in your own plugin! You don't want to break WordPress for your users.

Related to this, using `if (function_exists('NAME')) {` around all your functions and classes sounds like a great idea until you realize the fatal flaw. If something else has a function with the same name and their code loads first, your plugin will break. Using `if-exists` should be reserved for shared libraries only.

Remember: Good prefix names are unique and distinct to your plugin. This will help you and the next person in debugging, as well as prevent conflicts.

Analysis result:

```
# This plugin is using the prefixes "museder", "musederrestoreone", "musederrestoreonev2",  
"musederrestoreonepro", "musederrestoreoneadmin", "musederrestoreonerestore",  
"musederrestoreonereports", "musederrestoreoneadminui" for 167 element(s).
```

```
# Looks like there are elements not using common prefixes.
```

```
includes/class-backup-jobs.php:963 add_option($key, $value, '', 'no');  
includes/class-backup-jobs.php:980 add_option($key, $value, '', 'no');
```

Note: Options and Transients must be prefixed.

This is really important because the options are stored in a shared location and under the name you have set. If two plugins use the same name for options, they will find an interesting conflict when trying to read information introduced by the other plugin.

Also, once your plugin has active users, changing the name of an option is going to be really tricky, so let's make it robust from the very beginning.

Example(s) from your plugin:

```
includes/class-backup-jobs.php:963 add_option($key, $value, '', 'no');  
includes/class-backup-jobs.php:980 add_option($key, $value, '', 'no');
```

👉 **Continue with the review process.**

Read this email thoroughly.

Please, take the time to fully understand the issues we've raised. Review the examples provided, read the relevant documentation, and research as needed. Our goal is for you to gain a clear understanding of the problems so you can address them effectively and avoid similar issues when maintaining your plugin in the future.

Note that there may be false positives – we are humans and make mistakes, we apologize if there is anything we have gotten wrong. If you have doubts you can ask us for clarification, when asking us please be clear, concise, direct and include an example.

📋 **Complete your checklist.**

- ✓ I fixed all the issues in my plugin based on the feedback I received and my own review, as I know that the Plugins Team may not share all cases of the same issue. I am familiar with tools such as Plugin Check, PHPCS + WPCS, and similar utilities to help me identify problems in my code.
- ✓ I tested my updated plugin on a clean WordPress installation with [WP_DEBUG set to true](#).

⚠ **Do not skip this step.** Testing is essential to make sure your fixes actually work and that you haven't introduced new issues.

- ✓ I acknowledge that this review will be rejected if I overlook the issues or fail to test my code.
- ✓ I went to ["Add your plugin"](#) and uploaded the updated version. *I can continue updating the code there throughout the review process — the team will always check the latest version.*
- ✓ I replied to this email. I was concise and shared any clarifications or important context that the team needed to know.
I didn't list all the changes, as the team will review the entire plugin again and that is not necessary at all.

i To make this process as quick as possible and to avoid burden on the volunteers devoting their time to review this plugin's code, we ask you to thoroughly check all shared issues and fix them before sending the code back to us. *I know we already asked you to do so, and it is because we are really trying to make it very clear.*

While we try to make our reviews as exhaustive as possible we, like you, are humans and may have missed things. We appreciate your patience and understanding.

Review ID: R museder-restoreone/artherslin/1Dec25/T9 17Mar26/3.8

--

WordPress Plugins Team | plugins@wordpress.org

<https://make.wordpress.org/plugins/>

<https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/>

<https://wordpress.org/plugins/plugin-check/>

On Tue, Mar 10, 2026 at 6:18 PM UTC, WordPress.org Plugin Directory <plugins@wordpress.org> wrote:
This is an automated message to confirm that we have received your updated plugin file.

File updated by artherslin, version 2.7.243.

Comment: ****Subject:**** Museder RestoreOne – Updated version 2.7.243 submitted for review

Dear Plugin Review Team,

Thank you for your previous feedback on Museder RestoreOne. I have addressed the issues raised and am submitting ****version 2.7.243**** for review.

****Summary of compliance in this version:****

- ****Plugin URI**** is reachable: <https://musederlabs.com/restoreone-plugin/>
- ****External services**** are documented in the readme (== External services ==): Amazon S3 and OpenAI, with purpose, data sent, when, and links to Terms of Service and Privacy Policy. The plugin does not use external services by default.
- ****Security:**** All relevant handlers use nonce verification and capability checks; input is sanitized/validated and output is escaped. Direct file access is guarded with ``if ( ! defined( 'ABSPATH' ) ) exit;`` where appropriate.
- ****Core loading:**** No direct inclusion of wp-load.php, wp-config.php, or wp-blog-header.php. Required core includes (e.g. wp-admin/includes/file.php) are loaded only inside functions when needed.
- ****Paths:**** Plugin uses ``plugin_dir_path()``, ``plugin_dir_url()``, and ``wp_upload_dir()`` for paths and storage; user-generated files are stored under the uploads directory.
- ****PHP limits:**** ``set_time_limit()`` is used only inside long-running functions (backup/restore), not in init or global scope.
- ****Input handling:**** The plugin no longer processes the whole request body via ``file_get_contents('php://input')``; only the necessary fields are read and validated.
- ****Options/transients:**** Option names use a clear, statically visible prefix (e.g. ``museder_restoreone_job_lock_`` via a constant).
- ****Plugin Check:**** The code has been checked with the Plugin Check plugin's PHPCS ruleset (plugin-review.xml), resulting in ****0 errors and 0 warnings****.

I have tested the updated plugin on a clean WordPress installation with WP_DEBUG enabled. The latest code has been uploaded to the plugin directory for your review.

Thank you for your time. I am happy to provide any further information or clarification you need.

Best regards,
Jerry Lin
Author of Museder RestoreOne

https://wordpress.org/plugins/files/2025/11/10_18-18-53_museder-restoreone.zip

--

WordPress Plugins Team | plugins@wordpress.org

<https://make.wordpress.org/plugins/>

<https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/>

<https://wordpress.org/plugins/plugin-check/>

On Sat, Feb 7, 2026 at 12:42 PM UTC, WordPress.org Plugin Directory <plugins@wordpress.org> wrote:



Thank you for the changes "artherslin"!

Your plugin is not yet ready to be approved, you are receiving this email because the volunteers have manually checked it and have found some issues in the code / functionality of your plugin.

Please check this email thoroughly, address any issues listed, test your changes, and upload a corrected version of your code if all is well.

List of issues found

The URL(s) declared in your plugin seems to be invalid or does not work.

From your plugin:

Plugin URI: <https://musederlabs.com/> - museder-restoreone.php - Connection timed out after 5001 milliseconds

Undocumented use of a 3rd Party / external service

Plugins are permitted to require the use of third party/external services as long as they are clearly documented.

When your plugin reach out to external services, you must disclose it. This is true even if you are the one providing that service.

You are required to document it in a clear and plain language, so users are aware of: **what data is sent, why, where and under which conditions.**

To do this, you must update your readme file to **clearly explain that your plugin relies on third party/external services**, and include **at least the following information** for each third party/external service that this plugin uses:

- What the service is and what it is used for.
- What data is sent and when.
- Provide links to the service's terms of service and privacy policy.

Remember, this is for your own legal protection. Use of services must be upfront and well documented. This allows users to ensure that any legal issues with data transmissions are covered.

Example:

== External services ==

This plugin connects to an API to obtain weather information, it's needed to show the weather information and forecasts in the included widget.

It sends the user's location every time the widget is loaded (If the location isn't available and/or the user hasn't given their consent, it displays a configurable default location).

This service is provided by "PRT Weather INC": [terms of use](#), [privacy policy](#).

Example(s) from your plugin:

```
# Domain(s) mentioned in the readme file. Links to service terms and/or privacy policy not found.
includes/class-cloud-service.php:220 : sprintf( 'https://s3.%s.amazonaws.com', $settings['s3_region'] );
includes/class-backup-lite-s3-service.php:851 $host = 's3.' . $region . '.amazonaws.com';
includes/class-backup-lite-s3-service.php:849 $host = $bucket . '.s3.' . $region . '.amazonaws.com';
```

Calling core loading files directly

Calling core files like wp-config.php, wp-blog-header.php, wp-load.php directly via an include is not permitted.

These calls are prone to failure as not all WordPress installs have the exact same file structure. In addition it opens your plugin to security issues, as WordPress can be easily tricked into running code in an unauthenticated manner.

Your code should always exist in functions and be called by action hooks. This is true even if you need code to exist outside of WordPress. Code should only be accessible to people who are logged in and authorized, if it needs that kind of access. Your plugin's pages should be called via the dashboard like all the other settings panels, and in that way, they'll always have access to WordPress functions.

<https://developer.wordpress.org/plugins/hooks/>

There are some exceptions to the rule in certain situations and for certain core files. In that case, we expect you to use `require_once` to load them and to use a function from that file immediately after loading it.

If you are trying to "expose" an endpoint to be accessed directly by an external service, you have some options.

- You can expose a 'page' use [query_vars](#) and/or [rewrite rules](#) to create a virtual page which calls a function. [A practical example](#).
- You can create an [AJAX endpoint](#).
- You can create a [REST API endpoint](#).

Example(s) from your plugin:

```
includes/class-restore-handler.php:509 require_once ABSPATH . 'wp-admin/includes/media.php';
```

Determine files and directories locations correctly

WordPress provides several functions for easily determining where a given file or directory lives.

We detected that the way your plugin references some files, directories and/or URLs may not work with all WordPress setups. This happens because there are hardcoded references or you are using the WordPress internal constants.

Let's improve it, please check out the following documentation:

<https://developer.wordpress.org/plugins/plugin-basics/determining-plugin-and-content-directories/>

It contains all the functions available to determine locations correctly.

Most common cases in plugins can be solved using the following functions:

- For where your plugin is located: `plugin_dir_path()` , `plugin_dir_url()` , `plugins_url()`
- For the uploads directory: `wp_upload_dir()` (*Note: If you need to write files, please do so in a folder in the uploads directory, not in your plugin directories*).

Example(s) from your plugin:

```
includes/class-backup.php:4519 $prefixes[] = wp_normalize_path( trailingslashit( WP_LANG_DIR ) );
includes/class-settings.php:158 $user_file = trailingslashit( ABSPATH ) . 'wp-admin/includes/user.php';
includes/class-restore.php:300 $upgrade = rtrim( (string) ABSPATH, '/\\' ) . '/wp-admin/includes/upgrade.php';
templates/page-settings.php:24 $temp_dir = function_exists( 'get_temp_dir' ) ? get_temp_dir() : (
function_exists( 'sys_get_temp_dir' ) ? sys_get_temp_dir() : ABSPATH );
-----> ABSPATH
```

Don't Force Set PHP Limits Globally

While many plugins can need optimal settings for PHP, we ask you please not set them as global defaults.

Having defines like `ini_set('memory_limit', '-1')`; run globally (like on init or in the `__construct()` part of your code) means you'll be running that for everything on the site, which may cause your users to fall out of compliance with any limits or restrictions on their host.

If you must use those, you need to limit them specifically to only the exact functions that require them.

Example(s) from your plugin:

```
includes/class-ai1wm-converter.php:572 set_time_limit(600);
includes/class-restore-handler.php:2464 set_time_limit(0);
includes/class-ai1wm-converter.php:633 set_time_limit(600);
includes/class-restore-service.php:2714 set_time_limit(600);
includes/class-restore-handler.php:708 set_time_limit(600);
includes/class-restore-handler.php:300 set_time_limit(600);
includes/class-restore-service.php:291 set_time_limit(600);
includes/class-backup.php:4037 set_time_limit(0);
```

... out of a total of 10 incidences.

Nonces and User Permissions Needed for Security

Please add a nonce check to your input calls (`$_POST`, `$_GET`, `$REQUEST`) to prevent unauthorized access.

If you use `wp_ajax_` to trigger submission checks, remember they also need a nonce check.



Checking permissions: Keep in mind, a nonce check alone is not bulletproof security. Do not rely on nonces for authorization purposes. When needed, use it together with `current_user_can()` in order to prevent users without the right permissions from accessing things they shouldn't.

Also make sure that the nonce logic is correct by making sure it cannot be bypassed. Checking the nonce with `current_user_can()` is great, but mixing it with other checks can make the condition more complex and, without realising it, bypassable, remember that **anything** can be sent through an input, **don't**

trust any input.

Keep performance in mind. Don't check for post submission outside of functions. Doing so means that the check will run on every single load of the plugin, which means that every single person who views any page on a site using your plugin will be checking for a submission. This will make your code slow and unwieldy for users on any high traffic site, leading to instability and eventually crashes.

The following links may assist you in development:

- <https://developer.wordpress.org/plugins/security/nonces/>
- <https://developer.wordpress.org/plugins/javascript/ajax/#nonce>
- <https://developer.wordpress.org/plugins/settings/settings-api/>

From your plugin:

```
includes/pro/reports-controller.php:203 Museder_Restoreone_Reports_Controller::handle_report_
download() [classMethod] No nonce check found validating input origin on lines 203–218
# ↳ Line 210: $file = sanitize_text_field( wp_unslash( $_GET['file'] ?? '' ) );
includes/class-restore-handler.php:1574 Museder_Restoreone_Restore_Handler::chunk_finalize()
[classMethod] No nonce check found validating input origin on lines 1574–1587
# ↳ Line 1579: $session_id = isset( $_POST['session_id'] ) ? sanitize_text_field( wp_unslash(
$_POST['session_id'] ) ) : '';
includes/class-log-handler.php:246 Museder_Restoreone_Log_Handler::ajax_download_log()
[classMethod] No nonce check found validating input origin on lines 246–255
# ↳ Line 252: $log = isset( $_GET['log'] ) ? sanitize_text_field( wp_unslash( $_GET['log'] ) ) : '';
# ↳ Line 255: $log = sanitize_text_field( wp_unslash( $_GET['amp;log'] ) );
includes/class-restore-handler.php:1363 Museder_Restoreone_Restore_Handler::chunk_prepare()
[classMethod] No nonce check found validating input origin on lines 1363–1409
# ↳ Line 1370: $filename = sanitize_file_name( wp_unslash( $_POST['filename'] ) );
# ↳ Line 1376: $filesize = absint( wp_unslash( $_POST['filesize'] ) );
# ↳ Line 1382: $chunk_size = absint( wp_unslash( $_POST['chunk_size'] ) );
# ↳ Line 1388: $total_chunks = absint( wp_unslash( $_POST['total_chunks'] ) );
includes/class-ui.php:831 Museder_Restoreone_UI::handle_delete_backup() [classMethod] No nonce
check found validating input origin on lines 831–841
# ↳ Line 835: $filename = isset( $_POST['filename'] ) ? sanitize_text_field( wp_unslash(
$_POST['filename'] ) ) : '';
```

... out of a total of 9 incidences.

Please, make sure that the nonce logic is correct. It's important to be cautious when structuring conditional checks around nonces.

```
includes/ai/class-ai-rest-controller.php:79 if ( '' === $nonce || ! wp_verify_nonce( $nonce, 'wp_rest' ) )
{
```

Processing the whole input

We strongly recommend you **never** attempt to process the whole `$_POST`/`$_REQUEST`/`$_GET` stack. This makes your plugin slower as you're needlessly cycling through data you don't need. Instead, you should only be attempting to process the items within that are required for your plugin to function.

Example(s) from your plugin:

```
includes/class-settings.php:248 $data = file_get_contents( 'php://input' );
includes/class-schedule-handler.php:966 $raw = file_get_contents( 'php://input' ); // Fallback for
JSON payload.
```


Generic function/class/define/namespace/option names

All plugins must have unique function names, namespaces, defines, class and option names. This prevents your plugin from conflicting with other plugins or themes. We need you to update your plugin to use more unique and distinct names.

A good way to do this is with a prefix. For example, if your plugin is called "Museder RestoreOne" then you could use names like these:

- `function musere_save_post(){ ... }`
- `class MUSERE_Admin { ... }`
- `update_option( 'musere_options', $options );`
- `add_shortcode( 'musere_shortcode', $callback );`
- `register_setting( 'musere_settings', 'musere_user_id', ... );`
- `define( 'MUSERE_PLUGIN_DIR', plugin_dir_path( __FILE__ ) );`
- `global $musere_options;`
- `add_action( 'wp_ajax_musere_save_data', ... );`
- `namespace artherslin\musederrestoreone;`

Disclaimer: These are just examples that may have been self-generated from your plugin name, we trust you can find better options. If you have a good alternative, please use it instead, this is just an example.

The prefix should be **at least four (4) characters long** (don't try to use two- or three-letter prefixes anymore). We host almost 100,000 plugins on WordPress.org alone. There are tens of thousands more outside our servers. Believe us, you're likely to encounter conflicts.

You also need to avoid the use of `__` (double underscores), `wp_`, or `_` (single underscore) as a prefix. Those are reserved for WordPress itself. You can use them inside your classes, but not as stand-alone function.

Please remember, if you're using `_n()` or `__()` for translation, that's fine. We're **only** talking about functions you've created for your plugin, not the core functions from WordPress. In fact, those core features are why you need to not use those prefixes in your own plugin! You don't want to break WordPress for your users.

Related to this, using `if (!function_exists('NAME')) {` around all your functions and classes sounds like a great idea until you realize the fatal flaw. If something else has a function with the same name and their code loads first, your plugin will break. Using `if-exists` should be reserved for shared libraries only.

Remember: Good prefix names are unique and distinct to your plugin. This will help you and the next person in debugging, as well as prevent conflicts.

Analysis result:

```
# This plugin is using the prefixes "museder", "musederrestoreone", "musederrestoreonev2",
"musederrestoreonepro", "musederrestoreoneadmin", "musederrestoreonerestore",
"musederrestoreonereports", "musederrestoreoneadminui" for 169 element(s).
```

```
# Looks like there are elements not using common prefixes.
```

```
includes/class-backup-jobs.php:962 add_option($key, $value, '', 'no');
includes/class-backup-jobs.php:979 add_option($key, $value, '', 'no');
```

Note: Options and Transients must be prefixed.

This is really important because the options are stored in a shared location and under the name you have set. If two plugins use the same name for options, they will find an interesting conflict when trying to read information introduced by the other plugin.

Also, once your plugin has active users, changing the name of an option is going to be really tricky, so let's make it robust from the very beginning.

Example(s) from your plugin:

```
includes/class-backup-jobs.php:979 add_option($key, $value, '', 'no');  
includes/class-backup-jobs.php:962 add_option($key, $value, '', 'no');
```

👉 Continue with the review process.

Read this email thoroughly.

Please, take the time to fully understand the issues we've raised. Review the examples provided, read the relevant documentation, and research as needed. Our goal is for you to gain a clear understanding of the problems so you can address them effectively and avoid similar issues when maintaining your plugin in the future.

Note that there may be false positives – we are humans and make mistakes, we apologize if there is anything we have gotten wrong. If you have doubts you can ask us for clarification, when asking us please be clear, concise, direct and include an example.

📋 Complete your checklist.

✓ I fixed all the issues in my plugin based on the feedback I received and my own review, as I know that the Plugins Team may not share all cases of the same issue. I am familiar with tools such as Plugin Check, PHPCS + WPCS, and similar utilities to help me identify problems in my code.

✓ I tested my updated plugin on a clean WordPress installation with [WP_DEBUG set to true](#).

⚠ Do not skip this step. Testing is essential to make sure your fixes actually work and that you haven't introduced new issues.

✓ I acknowledge that this review will be rejected if I overlook the issues or fail to test my code.

✓ I went to ["Add your plugin"](#) and uploaded the updated version. *I can continue updating the code there throughout the review process — the team will always check the latest version.*

✓ I replied to this email. I was concise and shared any clarifications or important context that the team needed to know.

I didn't list all the changes, as the team will review the entire plugin again and that is not necessary at all.

📖 To make this process as quick as possible and to avoid burden on the volunteers devoting their time to review this plugin's code, we ask you to thoroughly check all shared issues and fix them before sending the code back to us. *I know we already asked you to do so, and it is because we are really trying to make it very clear.*

While we try to make our reviews as exhaustive as possible we, like you, are humans and may have missed things. We appreciate your patience and understanding.

Review ID: R museder-restoreone/artherslin/1Dec25/T8 7Feb26/3.8RC3

--

WordPress Plugins Team | plugins@wordpress.org

<https://make.wordpress.org/plugins/>

<https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/>

<https://wordpress.org/plugins/plugin-check/>

On Tue, Feb 3, 2026 at 5:15 PM UTC, WordPress.org Plugin Directory <plugins@wordpress.org> wrote:

This is an automated message to confirm that we have received your updated plugin file.

File updated by artherslin, version 2.7.241.

Comment: Subject: Re: [WordPress Plugin Directory] Review in Progress: Museder RestoreOne (Review ID: R museder-restoreone/artherslin/1Dec25/T7 23Jan26/3.8RC2)

Hello WordPress Plugins Team,

Thank you for the detailed feedback. I have addressed the issues and uploaded an updated version. Here is my reply:

--- Issues addressed ---

1. AI generated output

I have removed the file you mentioned (docs/wp-compliance-checklist.md). In the latest package, the entire docs/ folder has been excluded from the plugin zip so that no development/AI-generated documentation is shipped to users.

2. Generic function/class/define/namespace/option names

- All options, transients, and cron hook names now use the prefix "museder_restoreone_" (or the same in constants/classes where applicable).
- The previous "backup_lite" identifiers are only referenced in the main plugin file for one-time migration (renaming old options/crons to the new prefixed names and clearing old cron hooks). No new options or hooks use "backup_lite".
- For class-backup-jobs.php lines 962 and 979: the \$key passed to add_option() is built by get_option_lock_key(), which returns 'museder_restoreone_job_lock_' . sanitize_key(\$job_id). So the option name is prefixed; static analysis may not trace the variable.

3. Direct file access

All PHP files that can execute when accessed directly now include the ABSPATH check at the top (after <?php / namespace, before other code), including download-handler.php.

--- Checklist ---

- ✓ I fixed all the issues based on your feedback and my own review, and used Plugin Check / PHPCS where helpful.
- ✓ I tested the updated plugin on a clean WordPress installation with WP_DEBUG set to true.
- ✓ I acknowledge that the review may be rejected if issues are overlooked or testing is skipped.
- ✓ I have uploaded the updated version via "Add your plugin"; the team will see the latest code.
- ✓ I am replying to this email with the above clarifications.

Thank you for your time. I am happy to provide more detail or adjust anything if needed.

Best regards,
Jerry Lin (artherslin)

https://wordpress.org/plugins/files/2025/11/03_17-15-46_museder-restoreone.zip

--

WordPress Plugins Team | plugins@wordpress.org

<https://make.wordpress.org/plugins/>

<https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/>

<https://wordpress.org/plugins/plugin-check/>

On Fri, Jan 23, 2026 at 7:10 PM UTC, WordPress.org Plugin Directory <plugins@wordpress.org> wrote:



Thank you for the changes "artherslin"!

Your plugin is not yet ready to be approved, you are receiving this email because the volunteers have manually checked it and have found some issues in the code / functionality of your plugin.

Please check this email thoroughly, address any issues listed, test your changes, and upload a corrected version of your code if all is well.

List of issues found

AI generated output.

We've detected files that appear to be the output of an AI tool used during development.

These files usually include explanations or summaries of the changes made by the AI. That's great! We appreciate the effort you're putting into improving your plugin. However, those files aren't necessary for the plugin functionality.

They are also typically not useful as documentation because they tend to be partial and unconnected files that are also not maintained or updated.

We encourage you to use the available tools to improve your plugin. However, please remove any unnecessary AI-generated output. This will help keep the plugin package clean and reduce potential confusion for users and reviewers.

From your plugin:

`museder-restoreone/docs/wp-compliance-checklist.md`

Generic function/class/define/namespace/option names

All plugins must have unique function names, namespaces, defines, class and option names. This prevents your plugin from conflicting with other plugins or themes. We need you to update your plugin to use more unique and distinct names.

A good way to do this is with a prefix. For example, if your plugin is called "Museder RestoreOne" then you could use names like these:

- `function musere_save_post(){ ... }`
- `class MUSERE_Admin { ... }`
- `update_option( 'musere_options', $options );`
- `add_shortcode( 'musere_shortcode', $callback );`
- `register_setting( 'musere_settings', 'musere_user_id', ... );`
- `define( 'MUSERE_PLUGIN_DIR', plugin_dir_path( __FILE__ ) );`
- `global $musere_options;`
- `add_action('wp_ajax_musere_save_data', ... );`
- `namespace artherslin\musederrestoreone;`

Disclaimer: These are just examples that may have been self-generated from your plugin name, we trust you can find better options. If you have a good alternative, please use it instead, this is just an example.

The prefix should be **at least four (4) characters long** (don't try to use two- or three-letter prefixes anymore). We host almost 100,000 plugins on WordPress.org alone. There are tens of thousands more outside our servers. Believe us, you're likely to encounter conflicts.

You also need to avoid the use of `__` (double underscores), `wp_`, or `_` (single underscore) as a prefix. Those are reserved for WordPress itself. You can use them inside your classes, but not as stand-alone function.

Please remember, if you're using `_n()` or `__()` for translation, that's fine. We're **only** talking about functions you've created for your plugin, not the core functions from WordPress. In fact, those core features are why you need to not use those prefixes in your own plugin! You don't want to break WordPress for your users.

Related to this, using `if (!function_exists('NAME')) {` around all your functions and classes sounds like a great idea until you realize the fatal flaw. If something else has a function with the same name and their code loads first, your plugin will break. Using `if-exists` should be reserved for shared libraries only.

Remember: Good prefix names are unique and distinct to your plugin. This will help you and the next person in debugging, as well as prevent conflicts.

Analysis result:

```
# This plugin is using the prefixes "museder", "musederrestoreone", "musederrestoreonev2",
"musederrestoreonepro", "musederrestoreoneadmin", "musederrestoreonereports",
"musederrestoreonerestore", "musederrestoreoneadminui" for 22 element(s).
```

```
# This plugin is using the prefix "backup_lite" for 146 element(s).
```

```
# Looks like there are elements not using common prefixes.
```

```
includes/class-backup-jobs.php:962 add_option($key, $value, '', 'no');
```

```
includes/class-backup-jobs.php:979 add_option($key, $value, '', 'no');
```

Note: Options and Transients must be prefixed.

This is really important because the options are stored in a shared location and under the name you have set. If two plugins use the same name for options, they will find an interesting conflict when trying to read information introduced by the other plugin.

Also, once your plugin has active users, changing the name of an option is going to be really tricky, so let's make it robust from the very beginning.

Example(s) from your plugin:

```
includes/class-backup-jobs.php:962 add_option($key, $value, '', 'no');
```

```
includes/class-backup-jobs.php:979 add_option($key, $value, '', 'no');
```

Allowing direct file access to plugin files

Direct file access occurs when someone directly queries a PHP file. This can be done by entering the complete path to the file in the browser's URL bar or by sending a POST request directly to the file.

For files that only contain class or function definitions, the risk of something funky happening when accessed directly is minimal. However, for files that contain executable code (e.g., function calls, class instance creation, class method calls, or inclusion of other PHP files), the risk of security issues is hard to predict because it depends on the specific case, but it can exist and it can be high.

You can easily prevent this by adding the following code at the beginning of all PHP files that could potentially execute code if accessed directly:

```
if ( ! defined( 'ABSPATH' ) ) exit; // Exit if accessed directly
```

Add it after the <?php opening tag and after the namespace declaration, if any, but before any other code.

Example(s) from your plugin:

```
download-handler.php:19
```

👉 **Continue with the review process.**

Read this email thoroughly.

Please, take the time to fully understand the issues we've raised. Review the examples provided, read the relevant documentation, and research as needed. Our goal is for you to gain a clear understanding of the problems so you can address them effectively and avoid similar issues when maintaining your plugin in the future.

Note that there may be false positives – we are humans and make mistakes, we apologize if there is anything we have gotten wrong. If you have doubts you can ask us for clarification, when asking us please be clear, concise, direct and include an example.

Complete your checklist.

✓ I fixed all the issues in my plugin based on the feedback I received and my own review, as I know that the Plugins Team may not share all cases of the same issue. I am familiar with tools such as Plugin Check, PHPCS + WPCS, and similar utilities to help me identify problems in my code.

✓ I tested my updated plugin on a clean WordPress installation with [WP_DEBUG](#) set to true.

⚠ **Do not skip this step.** Testing is essential to make sure your fixes actually work and that you haven't introduced new issues.

✓ I acknowledge that this review will be rejected if I overlook the issues or fail to test my code.

✓ I went to **"Add your plugin"** and uploaded the updated version. *I can continue updating the code there throughout the review process — the team will always check the latest version.*

✓ I replied to this email. I was concise and shared any clarifications or important context that the team needed to know.

I didn't list all the changes, as the team will review the entire plugin again and that is not necessary at all.

 To make this process as quick as possible and to avoid burden on the volunteers devoting their time to review this plugin's code, we ask you to thoroughly check all shared issues and fix them before sending the code back to us. *I know we already asked you to do so, and it is because we are really trying to make it very clear.*

While we try to make our reviews as exhaustive as possible we, like you, are humans and may have missed things. We appreciate your patience and understanding.

Review ID: R museder-restoreone/artherslin/1Dec25/T7 23Jan26/3.8RC2

--

WordPress Plugins Team | plugins@wordpress.org

<https://make.wordpress.org/plugins/>

<https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/>

<https://wordpress.org/plugins/plugin-check/>

On Thu, Jan 15, 2026 at 9:57 AM UTC, WordPress.org Plugin Directory <plugins@wordpress.org> wrote:

This is an automated message to confirm that we have received your updated plugin file.

File updated by artherslin, version 2.7.241.

Comment: Subject: Re: Plugin Review – museder-restoreone – Updated submission (v2.7.241)

Hi Plugins Team,

Thank you for your review and detailed feedback. We have uploaded an updated version of Museder RestoreOne (v2.7.241).

- 1) We added clear documentation of external services in readme.txt (purpose, what data is sent, when it is sent, and links to Terms/Privacy).
- 2) We removed any direct loading of WordPress core files via ABSPATH and ensured code execution happens through WordPress hooks/AJAX/REST within the authenticated admin context.

- 3) We updated file/path handling to rely on WordPress APIs, and store generated files under `wp_upload_dir()`.
- 4) We ensured all sensitive actions/endpoints have proper capability checks (e.g., `current_user_can()`).
- 5) We implemented nonce validation for relevant requests where appropriate.
- 6) We applied proper sanitization/validation on inputs and escaping on outputs.
- 7) We standardized all plugin-defined function/class/constant/option/transient names with unique prefixes to avoid conflicts.
- 8) We added ABSPATH guards to prevent direct access to plugin PHP files.
- 9) We addressed the previously flagged unsafe SQL patterns by using WordPress database APIs and prepared queries where applicable.

We also tested the updated plugin on a clean WordPress installation with WP_DEBUG enabled and verified backup/restore workflows.

Thank you again for your time. Please let us know if you need any further clarification.

Kind regards,
Jerry Lin
Museder RestoreOne

https://wordpress.org/plugins/files/2025/11/15_09-57-18_museder-restoreone.zip

--

WordPress Plugins Team | plugins@wordpress.org

<https://make.wordpress.org/plugins/>

<https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/>

<https://wordpress.org/plugins/plugin-check/>

On Sun, Jan 11, 2026 at 4:44 PM UTC, WordPress.org Plugin Directory <plugins@wordpress.org> wrote:



Thank you for the changes "artherslin"!

Your plugin is not yet ready to be approved, you are receiving this email because the volunteers have manually checked it and have found some issues in the code / functionality of your plugin.

Please check this email thoroughly, address any issues listed, test your changes, and upload a corrected version of your code if all is well.

List of issues found

Undocumented use of a 3rd Party / external service

Plugins are permitted to require the use of third party/external services as long as they are clearly documented.

When your plugin reach out to external services, you must disclose it. This is true even if you are the one providing that service.

You are required to document it in a clear and plain language, so users are aware of: **what data is sent, why, where and under which conditions.**

To do this, you must update your readme file to clearly explain that your plugin relies on third party/external services, and include at least the following information for each third party/external service that this plugin uses:

- What the service is and what it is used for.
- What data is sent and when.

- Provide links to the service's terms of service and privacy policy.

Remember, this is for your own legal protection. Use of services must be upfront and well documented. This allows users to ensure that any legal issues with data transmissions are covered.

Example:

== External services ==

This plugin connects to an API to obtain weather information, it's needed to show the weather information and forecasts in the included widget.

It sends the user's location every time the widget is loaded (If the location isn't available and/or the user hasn't given their consent, it displays a configurable default location). This service is provided by "PRT Weather INC": [terms of use](#), [privacy policy](#).

Example(s) from your plugin:

Domain(s) mentioned in the readme file. Links to service terms and/or privacy policy not found.

```
includes/class-backup-lite-s3-service.php:849 $host = $bucket . '.s3.' . $region .
```

```
'amazonaws.com';
```

```
includes/class-backup-lite-s3-service.php:851 $host = 's3.' . $region . 'amazonaws.com';
```

```
includes/class-cloud-service.php:220 : sprintf( 'https://s3.%s.amazonaws.com',
```

```
$settings['s3_region'] );
```

Calling core loading files directly

Calling core files like wp-config.php, wp-blog-header.php, wp-load.php directly via an include is not permitted.

These calls are prone to failure as not all WordPress installs have the exact same file structure. In addition it opens your plugin to security issues, as WordPress can be easily tricked into running code in an unauthenticated manner.

Your code should always exist in functions and be called by action hooks. This is true even if you need code to exist outside of WordPress. Code should only be accessible to people who are logged in and authorized, if it needs that kind of access. Your plugin's pages should be called via the dashboard like all the other settings panels, and in that way, they'll always have access to WordPress functions.

<https://developer.wordpress.org/plugins/hooks/>

There are some exceptions to the rule in certain situations and for certain core files. In that case, we expect you to use `require_once` to load them and to use a function from that file immediately after loading it.

If you are trying to "expose" an endpoint to be accessed directly by an external service, you have some options.

- You can expose a 'page' use [query_vars](#) and/or [rewrite rules](#) to create a virtual page which calls a function. [A practical example](#).
- You can create an [AJAX endpoint](#).
- You can create a [REST API endpoint](#).

Example(s) from your plugin:

```
includes/class-backup.php:3846 require_once ABSPATH . 'wp-admin/includes/class-pclzip.php';
```

```
includes/class-restore-service.php:637 require_once ABSPATH . 'wp-includes/cron.php';
```

```
includes/class-backup.php:368 require_once ABSPATH . 'wp-admin/includes/class-
pclzip.php';
includes/class-ai1wm-converter.php:522 require_once ABSPATH . 'wp-admin/includes/class-
pclzip.php';
includes/class-restore-jobs.php:270 require_once ABSPATH . 'wp-includes/cron.php';
includes/class-ai1wm-converter.php:600 require_once ABSPATH . 'wp-admin/includes/class-
pclzip.php';
includes/class-chunk-handler.php:1066 require_once ABSPATH . 'wp-admin/includes/class-
pclzip.php';
includes/class-restore-service.php:2787 require_once ABSPATH . 'wp-admin/includes/class-
pclzip.php';
```

... out of a total of 12 incidences.

Determine files and directories locations correctly

WordPress provides several functions for easily determining where a given file or directory lives.

We detected that **the way your plugin references some files, directories and/or URLs may not work with all WordPress setups**. This happens because there are hardcoded references or you are using the WordPress internal constants.

Let's improve it, please check out the following documentation:

<https://developer.wordpress.org/plugins/plugin-basics/determining-plugin-and-content-directories/>

It contains all the functions available to determine locations correctly.

Most common cases in plugins can be solved using the following functions:

- For where your plugin is located: `plugin_dir_path()` , `plugin_dir_url()` , `plugins_url()`
- For the uploads directory: `wp_upload_dir()` (*Note: If you need to write files, please do so in a folder in the uploads directory, not in your plugin directories*).

Example(s) from your plugin:

```
includes/helpers.php:793 $instruction = "location ~ ^" . trailingslashit( str_replace( ABSPATH,
'/', wp_normalize_path( $dir ) ) ) . " {\n    deny all;\n}\n";
-----> ABSPATH
includes/class-restore-service.php:1212 [ 'base' => dirname( WP_CONTENT_DIR ), 'include'
=> [ 'wp-content' ] ],
includes/class-backup.php:4738 trailingslashit( WP_CONTENT_DIR . '/wp-rocket-cache' ),
includes/pro/ai-service.php:373 $wp_content = WP_CONTENT_DIR;
includes/class-restore-service.php:1504 $sites_dir = wp_normalize_path( trailingslashit(
WP_CONTENT_DIR ) . 'uploads/sites' );
-----> WP_CONTENT_DIR
includes/class-estimate-size.php:131 $wp_content_path = WP_CONTENT_DIR;
includes/class-backup.php:4819 $line = wp_normalize_path( WP_CONTENT_DIR . '/' . substr(
$line, strlen( 'wp-content/' ) ) );
-----> WP_CONTENT_DIR
includes/class-restore-service.php:1516 $dest = wp_normalize_path( trailingslashit(
WP_CONTENT_DIR ) . 'uploads' );
```

... out of a total of 35 incidences.

 If your plugin creates files, those should be saved in the uploads directory.

In order to determine that directory location you can make use of the `wp_upload_dir()` function.

Also, it's a good practice that you store those files under a folder with the same name as your plugin

slug "museder-restoreone"

Example:

```
$path = wp_upload_dir()['basedir'] . '/museder-restoreone';
```

Example(s) from your plugin:

```
includes/class-restore-service.php:1544 $sites_dir = wp_normalize_path( trailingslashit(
WP_CONTENT_DIR ) . 'uploads/sites' );
-----> 'uploads/sites'
includes/class-restore-service.php:1211 [ 'base' => WP_CONTENT_DIR, 'include' => [
'uploads', 'plugins', 'themes', 'mu-plugins' ] ],
-----> 'uploads'
```

... out of a total of 4 incidences.

 In order to determine your plugin location, you would need to use the `__FILE__` variable for this to work properly.

Note that this variable depends on the location of the file making the call. As this can create confusion, a common practice is to save its value in a `define()` in the main file of your plugin so that you don't have to worry about this.

Example: Your main plugin file.

```
define( 'MUSERE_PLUGIN_FILE', __FILE__ );
define( 'MUSERE_PLUGIN_DIR', plugin_dir_path( __FILE__ ) );
define( 'MUSERE_PLUGIN_URL', plugin_dir_url( __FILE__ ) );
```

Example: Any file of your plugin.

```
require_once MUSERE_PLUGIN_DIR . 'admin/class-init.php';
```

```
function musere_scripts() {
    wp_enqueue_script( 'musere-script', MUSERE_PLUGIN_URL . 'js/script.js', array(),
MUSERE_VERSION );
    // Or alternatively
    wp_enqueue_script( 'musere-script', plugins_url( 'js/script.js', MUSERE_PLUGIN_FILE ),
array(), MUSERE_VERSION );
}
add_action( 'wp_enqueue_scripts', 'musere_scripts' );
```

Example(s) from your plugin:

```
includes/class-backup.php:1729 $plugins_dir = defined( 'WP_PLUGIN_DIR' ) ?
wp_normalize_path( WP_PLUGIN_DIR ) : "";
-----> WP_PLUGIN_DIR
includes/class-backup.php:4584 $prefixes[] = wp_normalize_path( trailingslashit(
WP_PLUGIN_DIR ) );
```

... out of a total of 5 incidences.

Nonces and User Permissions Needed for Security

Please add a nonce check to your input calls (`$_POST`, `$_GET`, `$REQUEST`) to prevent

unauthorized access.

If you use `wp_ajax_` to trigger submission checks, remember they also need a nonce check.

 **Checking permissions:** Keep in mind, a nonce check alone is not bulletproof security. Do not rely on nonces for authorization purposes. When needed, use it together with `current_user_can()` in order to prevent users without the right permissions from accessing things they shouldn't.

Also make sure that the **nonce logic is correct by making sure it cannot be bypassed**.

Checking the nonce with `current_user_can()` is great, but mixing it with other checks can make the condition more complex and, without realising it, bypassable, remember that **anything** can be sent through an input, **don't trust any input**.

Keep performance in mind. Don't check for post submission outside of functions. Doing so means that the check will run on every single load of the plugin, which means that every single person who views any page on a site using your plugin will be checking for a submission. This will make your code slow and unwieldy for users on any high traffic site, leading to instability and eventually crashes.

The following links may assist you in development:

- <https://developer.wordpress.org/plugins/security/nonces/>
- <https://developer.wordpress.org/plugins/javascript/ajax/#nonce>
- <https://developer.wordpress.org/plugins/settings/settings-api/>

From your plugin:

```
includes/class-ui.php:900 Backup_Lite_UI::handle_delete_restore_history() [classMethod] No
nonce check found validating input origin on lines 900-911
# ↳ Line 906: $raw = wp_unslash( $_POST['timestamps'] );
includes/class-restore-handler.php:1349 Backup_Lite_Restore_Handler::chunk_upload()
[classMethod] No nonce check found validating input origin on lines 1349-1372
# ↳ Line 1356: $session_id = sanitize_text_field( wp_unslash( $_POST['session_id'] ) );
# ↳ Line 1362: $index = absint( wp_unslash( $_POST['chunk_index'] ) );
# ↳ Line 1387: if ( ! isset( $_FILES['chunk']['tmp_name'] ) || ! is_uploaded_file(
$_FILES['chunk']['tmp_name'] ) ) {
# ↳ Line 1405: $tmp_name = $_FILES['chunk']['tmp_name'];
includes/class-schedule-handler.php:130 Backup_Lite_Schedule_Handler::
ajax_delete_schedule() [classMethod] No nonce check found validating input origin on lines
130-144
# ↳ Line 139: $schedule_id = isset( $_POST['schedule_id'] ) ? sanitize_text_field( wp_unslash(
$_POST['schedule_id'] ) ) : '';
# ↳ Line 153: $nonce = isset( $_POST['_ajax_nonce'] ) ? wp_unslash( $_POST['_ajax_nonce'] )
: '';
includes/class-ui.php:1113 Backup_Lite_UI::get_backup_options_from_request() [classMethod]
No nonce check found validating input origin on lines 1113-1123
# ↳ Line 1120: $backup_mode = sanitize_key( wp_unslash( $_POST['backup_mode'] ) );
# ↳ Line 1128: $smart_exclude = sanitize_key( wp_unslash( $_POST['backup_smart_exclude'] )
);
# ↳ Line 1136: $custom_excludes = wp_unslash( $_POST['backup_custom_excludes'] );
# ↳ Line 1159: $val = wp_unslash( $_POST[ $post_key ] );
# ↳ Line 1171: $include_tables = wp_unslash( $_POST['include_db_tables'] );
# ↳ Line 1184: $exclude_tables = wp_unslash( $_POST['exclude_db_tables'] );
# ↳ Line 1197: $blog_id_raw = wp_unslash( $_POST['multisite_blog_id'] );
# ↳ Line 1207: $backup_label = sanitize_text_field( wp_unslash( $_POST['backup_label'] ) );
# ↳ Line 1225: $backup_cloud_raw = wp_unslash( $_POST['backup_cloud'] );
includes/class-restore-handler.php:1588 Backup_Lite_Restore_Handler::chunk_status()
[classMethod] No nonce check found validating input origin on lines 1588-1600
```

```
# ↳ Line 1593: $session_id = isset( $_POST['session_id'] ) ? sanitize_text_field( wp_unslash(
$_POST['session_id'] ) ) : '';
```

... out of a total of 18 incidences.

Please, make sure that the nonce logic is correct. It's important to be cautious when structuring conditional checks around nonces.

```
includes/ai/class-ai-rest-controller.php:79 if ( " === $nonce || ! wp_verify_nonce( $nonce,
'wp_rest' ) ) {
```

Data Must be Sanitized, Escaped, and Validated

When you include POST/GET/REQUEST/FILE calls in your plugin, it's important to sanitize, validate, and escape them. The goal here is to prevent a user from accidentally sending trash data through the system, as well as protecting them from potential security issues.

SANITIZE: Data that is input (either by a user or automatically) must be sanitized as soon as possible. This lessens the possibility of XSS vulnerabilities and MITM attacks where posted data is subverted.

VALIDATE: All data should be validated, no matter what. Even when you sanitize, remember that you don't want someone putting in 'dog' when the only valid values are numbers.

ESCAPE: Data that is output must be escaped properly when it is echo'd, so it can't hijack admin screens. There are many `esc_*`() functions you can use to make sure you don't show people the wrong data.

To help you with this, WordPress comes with a number of sanitization and escaping functions. You can read about those here:

- <https://developer.wordpress.org/apis/security/sanitizing/>
- <https://developer.wordpress.org/apis/security/escaping/>

Remember: You must use the most appropriate functions for the context. If you're sanitizing email, use `sanitize_email()`, if you're outputting HTML, use `wp_kses_post()`, and so on.

An easy mantra here is this:

Sanitize *early*

Escape *Late*

Always Validate

Clean everything, check everything, escape everything, and never trust the users to always have input sane data. After all, users come from all walks of life.

Example(s) from your plugin:

```
includes/class-schedule-handler.php:153 $nonce = isset( $_POST['_ajax_nonce'] ) ?
wp_unslash( $_POST['_ajax_nonce'] ) : '';
includes/class-schedule-handler.php:346 $nonce = isset( $_POST['_ajax_nonce'] ) ?
wp_unslash( $_POST['_ajax_nonce'] ) : '';
```

Note: While the `json_decode()` function in PHP is useful for decoding JSON strings, it does not sanitize the input. Sanitization refers to the process of cleaning or filtering the input data to ensure that it is safe and secure to use.

The `json_decode()` function simply transforms a JSON string into a PHP array or object. Any potentially malicious data or scripts may persist after `json_decode()`.

Example(s) from your plugin:


```
includes/class-ui.php:732 $decoded = json_decode( $search_replace_raw, true );
includes/class-schedule-handler.php:978 $decoded = json_decode( $raw_schedule, true );
includes/class-ui.php:911 $decoded = json_decode( wp_unslash( $raw ), true );
includes/class-ui.php:1233 $decoded = json_decode( $cloud, true );
```

... out of a total of 7 incidences.

✓ You can check this using [Plugin Check](#).

Generic function/class/define/namespace/option names

All plugins must have unique function names, namespaces, defines, class and option names. This prevents your plugin from conflicting with other plugins or themes. We need you to update your plugin to use more unique and distinct names.

A good way to do this is with a prefix. For example, if your plugin is called "Museder RestoreOne" then you could use names like these:

- function musere_save_post(){ ... }
- class MUSERE_Admin { ... }
- update_option('musere_options', \$options);
- add_shortcode('musere_shortcode', \$callback);
- register_setting('musere_settings', 'musere_user_id', ...);
- define('MUSERE_PLUGIN_DIR', plugin_dir_path(__FILE__));
- global \$musere_options;
- add_action('wp_ajax_musere_save_data', ...);
- namespace artherslin\musederrestoreone;

Disclaimer: These are just examples that may have been self-generated from your plugin name, we trust you can find better options. If you have a good alternative, please use it instead, this is just an example.

The prefix should be **at least four (4) characters long** (don't try to use two- or three-letter prefixes anymore). We host almost 100,000 plugins on WordPress.org alone. There are tens of thousands more outside our servers. Believe us, you're likely to encounter conflicts.

You also need to avoid the use of `__` (double underscores), `wp_`, or `_` (single underscore) as a prefix. Those are reserved for WordPress itself. You can use them inside your classes, but not as stand-alone function.

Please remember, if you're using `_n()` or `__()` for translation, that's fine. We're **only** talking about functions you've created for your plugin, not the core functions from WordPress. In fact, those core features are why you need to not use those prefixes in your own plugin! You don't want to break WordPress for your users.

Related to this, using `if ( !function_exists('NAME') ) {` around all your functions and classes sounds like a great idea until you realize the fatal flaw. If something else has a function with the same name and their code loads first, your plugin will break. Using `if-exists` should be reserved for shared libraries only.

Remember: Good prefix names are unique and distinct to your plugin. This will help you and the next person in debugging, as well as prevent conflicts.

Analysis result:

```
# This plugin is using the prefixes "museder", "musederrestoreone", "musederrestoreonev2",
"musederrestoreonepro", "musederrestoreoneadmin", "musederrestoreonereports",
"musederrestoreoneadminui", "musederrestoreonerestore" for 22 element(s).
# This plugin is using the prefix "backup_lite" for 148 element(s).
```

Looks like there are elements not using common prefixes.

```
includes/class-ui.php:289 wp_localize_script('backup-lite-admin', 'backupLiteAdmin',
```



```

['ajax_url' => admin_url('admin-ajax.php'), 'nonce' => wp_create_nonce('backup_lite_
admin_actions')]);
# ↳ Detected name: backupLiteAdmin
includes/class-ui.php:310 wp_localize_script('backup-lite-admin', 'backupLiteSchedulesL10n',
['ajaxUrl' => admin_url('admin-ajax.php'), 'nonce' => wp_create_nonce('backup_lite_
admin_actions'), 'updateSchedule' => __('Update Schedule', 'museder-restoreone'),
'saveSchedule' => __('Save Schedule', 'museder-restoreone'), 'confirmDelete' => __('Are you
sure you want to delete this schedule?', 'museder-restoreone')]);
# ↳ Detected name: backupLiteSchedulesL10n
includes/class-backup-jobs.php:902 add_option($key, $value, '', 'no');
includes/class-backup-jobs.php:919 add_option($key, $value, '', 'no');

```

Note: Options and Transients must be prefixed.

This is really important because the options are stored in a shared location and under the name you have set. If two plugins use the same name for options, they will find an interesting conflict when trying to read information introduced by the other plugin.

Also, once your plugin has active users, changing the name of an option is going to be really tricky, so let's make it robust from the very beginning.

Example(s) from your plugin:

```

includes/class-backup-jobs.php:919 add_option($key, $value, '', 'no');
includes/class-backup-jobs.php:902 add_option($key, $value, '', 'no');

```

Allowing direct file access to plugin files

Direct file access occurs when someone directly queries a PHP file. This can be done by entering the complete path to the file in the browser's URL bar or by sending a POST request directly to the file.

For files that only contain class or function definitions, the risk of something funky happening when accessed directly is minimal. However, for files that contain executable code (e.g., function calls, class instance creation, class method calls, or inclusion of other PHP files), the risk of security issues is hard to predict because it depends on the specific case, but it can exist and it can be high.

You can easily prevent this by adding the following code at the beginning of all PHP files that could potentially execute code if accessed directly:

```
if ( ! defined( 'ABSPATH' ) ) exit; // Exit if accessed directly
```

Add it after the <?php opening tag and after the namespace declaration, if any, but before any other code.

Example(s) from your plugin:

```
download-handler.php:14
```

Unsafe SQL calls

When making database calls, it's highly important to protect your code from SQL injection vulnerabilities. You need to update your code to use wpdb calls and prepare() with your queries to protect them.

Please review the following:

- <https://developer.wordpress.org/reference/classes/wpdb/#protect-queries-against-sql-injection-attacks>
- https://codex.wordpress.org/Data_Validation#Database
- <https://make.wordpress.org/core/2012/12/12/php-warning-missing-argument-2-for-wpdb-prepare/>
- <https://ottopress.com/2013/better-know-a-vulnerability-sql-injection/>

Example(s) from your plugin:

```
includes/class-restore.php:574 $batch[] = $tuples[ $i ];
includes/class-restore.php:563 $tuple = $tuples[ $i ];
includes/class-restore.php:578 $sql = $prefix . implode( ',', $batch ) . ';';
includes/class-restore.php:581 $result = $wpdb->query( $sql ); // phpcs:ignore
WordPress.DB.PreparedSQL.NotPrepared -- SQL from trusted backup file
-----> $wpdb->query($sql)
# You cannot add variables like "$tuples[$i]" directly to the SQL query.
# Using wpdb::prepare($query, $args) you will need to include placeholders for each variable
within the query and include the variables in the second parameter.
# You cannot add calls like "implode(',', $batch)" directly to the SQL query. You need to use
wpdb::prepare.
# Remember that using wpdb::prepare($query, $args) you will need to include placeholders for
each variable within the query and include the variables in the second parameter.
# The SQL query needs to be included in a wpdb::prepare($query, $args) function.
```

```
includes/class-restore.php:499 $prepared = (string) $prepared;
includes/class-restore.php:606 $result = $wpdb->query( $prepared ); // phpcs:ignore
WordPress.DB.PreparedSQL.NotPrepared -- SQL from trusted backup file
-----> $wpdb->query($prepared)
# The SQL query needs to be included in a wpdb::prepare($query, $args) function.
# Remember that you will need to include placeholders for each variable within the query and
include their calls in the second parameter of wpdb::prepare().
```

... out of a total of 4 incidences.

Note: Passing individual values to wpdb::prepare using placeholders is fairly straightforward, but what if we need to pass an array of values instead?

You'll need to create a placeholder for each item of the array and pass all the corresponding values to those placeholders, this seems tricky, but here is a snippet to do so.

```
$wordcamp_id_placeholders = implode( ' ', array_fill( 0, count( $wordcamp_ids ), '%d' ) );
$prepare_values = array_merge( array( $new_status ), $wordcamp_ids );
$wpdb->query( $wpdb->prepare( "
    UPDATE ` $table_name `
    SET `post_status` = %s
    WHERE ID IN ( $wordcamp_id_placeholders )",
    $prepare_values
) );
```

There is a core ticket that could make this easier in the future: <https://core.trac.wordpress.org/ticket/54042>

Example(s) from your plugin:

```
includes/class-restore-service.php:2398 'SELECT ' . implode( ',', $select_sql_cols ) . ' FROM
` . esc_sql( $safe_table ) . ` LIMIT %d OFFSET %d', // phpcs:ignore
WordPress.DB.PreparedSQL.NotPrepared, PluginCheck.Security.DirectDB.
UnescapedDBParameter -- identifiers sanitized + escaped; values prepared
-----> implode( ',', $select_sql_cols )
includes/class-restore-service.php:2374 'SELECT ' . implode( ',', $select_sql_cols ) . ' FROM `
```

```
. esc_sql( $safe_table ) . ` WHERE ` . $pk_sql . ` > %s ORDER BY ` . $pk_sql . ` ASC LIMIT
%d', // phpcs:ignore WordPress.DB.PreparedSQL.NotPrepared,
PluginCheck.Security.DirectDB.UnescapedDBParameter -- identifiers sanitized + escaped;
values prepared
-----> implode(',', $select_sql_cols)

... out of a total of 5 incidences.
```

👉 Continue with the review process.

Read this email thoroughly.

Please, take the time to fully understand the issues we've raised. Review the examples provided, read the relevant documentation, and research as needed. Our goal is for you to gain a clear understanding of the problems so you can address them effectively and avoid similar issues when maintaining your plugin in the future.

Note that there may be false positives – we are humans and make mistakes, we apologize if there is anything we have gotten wrong. If you have doubts you can ask us for clarification, when asking us please be clear, concise, direct and include an example.

📋 Complete your checklist.

- ✓ I fixed all the issues in my plugin based on the feedback I received and my own review, as I know that the Plugins Team may not share all cases of the same issue. I am familiar with tools such as Plugin Check, PHPCS + WPCS, and similar utilities to help me identify problems in my code.
- ✓ I tested my updated plugin on a clean WordPress installation with [WP_DEBUG set to true](#).

⚠️ **Do not skip this step.** Testing is essential to make sure your fixes actually work and that you haven't introduced new issues.

- ✓ I acknowledge that this review will be rejected if I overlook the issues or fail to test my code.
- ✓ I went to ["Add your plugin"](#) and uploaded the updated version. *I can continue updating the code there throughout the review process — the team will always check the latest version.*
- ✓ I replied to this email. I was concise and shared any clarifications or important context that the team needed to know.
I didn't list all the changes, as the team will review the entire plugin again and that is not necessary at all.

i To make this process as quick as possible and to avoid burden on the volunteers devoting their time to review this plugin's code, we ask you to **thoroughly check all shared issues and fix them before sending the code back to us**. *I know we already asked you to do so, and it is because we are really trying to make it very clear.*

While we try to make our reviews as exhaustive as possible we, like you, are humans and may have missed things. We appreciate your patience and understanding.

Review ID: R museder-restoreone/artherslin/1Dec25/T5 11Jan26/3.8RC1

--

WordPress Plugins Team | plugins@wordpress.org
<https://make.wordpress.org/plugins/>

<https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/>

<https://wordpress.org/plugins/plugin-check/>

On Wed, Jan 7, 2026 at 9:23 AM UTC, WordPress.org Plugin Directory

<plugins@wordpress.org> wrote:

This is an automated message to confirm that we have received your updated plugin file.

File updated by artherslin, version 2.7.222.

Comment: Hello Plugin Review Team,

Thank you for the detailed review.

We have addressed the reported items and uploaded an updated build v2.7.222 for re-review.

Invalid Plugin URI / Author URI: Updated both header URLs to valid, reachable WordPress.org profile URLs to resolve the TLS connection error.

External services disclosure: Added/expanded the == External services == section in readme.txt, including: what the service is used for, what data is sent and when, and links to Terms/Privacy.

We also explicitly mention S3 endpoints/domains (e.g., s3.{region}.amazonaws.com / {bucket}.s3.{region}.amazonaws.com).

Correct directory/path usage: Removed hard-coded references like

WP_CONTENT_DIR.'/uploads' / WP_CONTENT_DIR.'/plugins' and now rely on WordPress directory helpers (e.g., wp_upload_dir(), plugins_url(), etc.). All writable data is stored under wp-content/uploads/museder-restoreone/.

No production error reporting: Removed permanent error_reporting() behavior.

HTTP API: Replaced custom cURL usage with the WordPress HTTP API (wp_remote_request()).

Changing active plugins: Removed any code that modifies active_plugins (we do not activate/deactivate other plugins automatically). Any recovery/safe-mode behavior is user-driven and does not alter other plugins' activation state.

Nonces & permissions / sanitization: Hardened privileged entry points to use capability checks + proper nonce verification (using WordPress standard nonce helpers) and ensured inputs are sanitized/validated.

Thanks again for your time — happy to adjust further if anything remains.

https://wordpress.org/plugins/files/2025/11/07_09-23-26_museder-restoreone.zip

--

WordPress Plugins Team | plugins@wordpress.org

<https://make.wordpress.org/plugins/>

<https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/>

<https://wordpress.org/plugins/plugin-check/>